Name: Kamran Gul

CMS: 023-25-0161

Section: C

OOP Lab: 08

Task 1

Teacher and Laptop (Association)

Scenario:

A `Teacher` has a name and may **have a** `Laptop`. The laptop can exist without the teacher (e.g., in a store). The teacher can have no laptop or can change laptop. This is **Association**: both objects exist independently; we just link them.

Requirements (3 methods in total):

1. **Class `Laptop`:**
   - Attribute: `brand` (String).
   - Constructor: `Laptop(String brand)`.
   - **Method 1:** `getBrand()` - returns the brand name.
2. **Class `Teacher`:**
   - Attributes: `name` (String), `laptop` (Laptop reference, can be `null`).
   - Constructor: `Teacher(String name)` - set `laptop = null`.
   - **Method 2:** `setLaptop(Laptop l)` - sets the teacher's laptop.
   - **Method 3:** `displayInfo()` - prints name and, if `laptop` is not null, prints "Laptop: " + laptop.getBrand(); otherwise prints "No laptop."
3. **In `Main`:**
   - Create a `Laptop` and a `Teacher` **separately**.
   - Call `teacher.setLaptop(laptop)` to link them (association).
   - Call `teacher.displayInfo()`. Then set `teacher.setLaptop(null)` and call `displayInfo()` again to show optional link.

Solution

```
class Laptop{
	String brand;
	Laptop(String brand){
		this.brand = brand;
	}
	String getBrand(){
		return brand;
	}
}

class Teacher{
	String name;
	Laptop laptop;
	Teacher(String name){
		this.name = name;
	}
	void setLaptop(Laptop laptop){
		this.laptop = laptop;
	}
	void displayInfo(){
```

```
                if(laptop==null){
                        System.out.println("Name: "+this.name);
                        System.out.println("No laptop");
                }
                else{
                        System.out.println("Name: "+this.name);
                        System.out.println("Brand: "+ laptop.getBrand());
                }
        }
}

class Main{
        public static void main(String[] args){
                Teacher teacher = new Teacher("Saif Hassan");
                Laptop laptop = new Laptop("HP");
                System.out.println("\nMaking Association by setting a laptop for teacher");
                teacher.setLaptop(laptop);
                teacher.displayInfo();
                System.out.println("\nMaking Optional Association by setting a laptop to null");
                teacher.setLaptop(null);
                teacher.displayInfo();
        }
}
```

Output

```
Making Association by setting a laptop for teacher
Name: Saif Hassan
Brand: HP

Making Optional Association by setting a laptop to null
Name: Saif Hassan
No laptop
```

Task 2

Team and Player (Aggregation)

Scenario:

A `Team` has a name and **has many** `Players`. Players are created elsewhere and **added** to the team (e.g., signed from outside). If the team is dissolved, players still exist. This is **Aggregation**: whole-part, part can exist without whole. Use a **simple array** to store players and a counter for how many are added.

Requirements (3 methods in total):

1. **Class `Player`:**
   - Attribute: `name` (String).
   - Constructor: `Player(String name)`.
   - One getter `getName()` if you use it in display (or access via getter from Team's display method).
2. **Class `Team`:**
   - Attributes: `name` (String), `Player[] players` (simple array), `int count` (number of players added).
   - Constructor: `Team(String name, int maxPlayers)` - create the array: `players = new Player[maxPlayers]`, set `count = 0`.
   - **Method 1:** `addPlayer(Player p)` - if `count < players.length`, set `players[count] = p` and increment `count`.
   - **Method 2:** `getTeamName()` - returns the team name.
   - **Method 3:** `displayPlayers()` - loop from `0` to `count` and print each player's name.

3. **In `Main`:**
   - Create several `Player` objects.
   - Create one `Team` with a maximum size (e.g. 10).
   - Add players with `addPlayer(...)` (aggregation: parts added from outside).
   - Call `displayPlayers()` and `getTeamName()`.

Solution

```
class Player{
	String name;
	Player(String name){
		this.name = name;
	}
	String getName(){
		return this.name;
	}
}

class Team{
	String name;
	Player[] players;
	int count;
	Team(String name, int maxPlayers){
		this.name = name;
		players = new Player[maxPlayers];
		count = 0;
	}
	void addPlayer(Player p){
		if(count<players.length){
			players[count] = p;
			count++;
		}
	}
	String getTeamName(){
		return this.name;
	}
	void displayPlayers(){
		for(Player p: players){
			if(p!=null)
			System.out.println(p.name);
		}
	}
}

class Main{
	public static void main(String[] args){
		Player p1 = new Player("Suryakumar Yadav");
		Player p2 = new Player("Shubman Gill");
		Player p3 = new Player("Jasprit Bumrah");
		Player p4 = new Player("Rinku Singh");
		Player p5 = new Player("Hardik Pandya");
		Player p6 = new Player("Sanju Samson");
		Player p7 = new Player("Abhishek Sharma");
		Player p8 = new Player("Arshdeep Singh");
		Player p9 = new Player("Axar Patel");
		Player p10 = new Player("Kuldeep Yadav");
```

```
        Team team = new Team("Indian Cricket Team",10);
        team.addPlayer(p1);
        team.addPlayer(p2);
        team.addPlayer(p3);
        team.addPlayer(p4);
        team.addPlayer(p5);
        team.addPlayer(p6);
        team.addPlayer(p7);
        team.addPlayer(p8);
        team.addPlayer(p9);
        team.addPlayer(p10);

        System.out.println("\nTeam Name: "+ team.getTeamName());

        System.out.println("\nTeam Players:");
        team.displayPlayers();
    }
}
```

Output

```
Team Name: Indian Cricket Team

Team Players:
Suryakumar Yadav
Shubman Gill
Jasprit Bumrah
Rinku Singh
Hardik Pandya
Sanju Samson
Abhishek Sharma
Arshdeep Singh
Axar Patel
Kuldeep Yadav
```

Task 3

Computer and CPU (Composition)

Scenario:

A `Computer` has a brand and **has a** `CPU`. The CPU does not exist without the computer; it is **created inside** the Computer constructor. This is **Composition**: whole owns part; part is created by the whole.

Requirements (3 methods in total):

1. **Class `CPU`:**
   - Attribute: `model` (String), e.g. "Intel i5".
   - Constructor: `CPU(String model)`.
   - **Method 1:** `getModel()` - returns the CPU model.
2. **Class `Computer`:**
   - Attributes: `brand` (String), `cpu` (CPU).
   - Constructor: `Computer(String brand, String cpuModel)` - inside the constructor, create the CPU: `this.cpu = new CPU(cpuModel);` (composition: part created inside whole).
   - **Method 2:** `getComputerInfo()` - returns a string with brand and CPU model (use cpu.getModel()).

- **Method 3:** `start()` - prints "Computer started with CPU: " + cpu.getModel() (or similar).

3. **In `Main`:**
   - Create a `Computer` by giving brand and CPU model (e.g. `new Computer("Dell", "Intel i5")`). Do **not** create a `CPU` in Main; the Computer creates it.
   - Call `getComputerInfo()` and `start()` to show that the CPU is inside the computer.

Solution

```
class CPU{
	String model;
	CPU(String model){
		this.model = model;
	}
	String getModel(){
		return model;
	}
}

class Computer{
	String brand;
	CPU cpu;
	Computer(String brand, String model){
		this.brand = brand;
		this.cpu = new CPU(model);
	}
	String getComputerInfo(){
		return this.brand + " " + cpu.getModel();
	}
	void start(){
		System.out.println("Computer Started with CPU "+cpu.model);
	}
}

class Main{
	public static void main(String[] args){
		Computer computer = new Computer("HP","Intel Corei8");
		System.out.print("\n\n");
		System.out.println(computer.getComputerInfo());
		computer.start();
	}
}
```

Output

```
HP Intel Corei8
Computer Started with CPU Intel Corei8
```